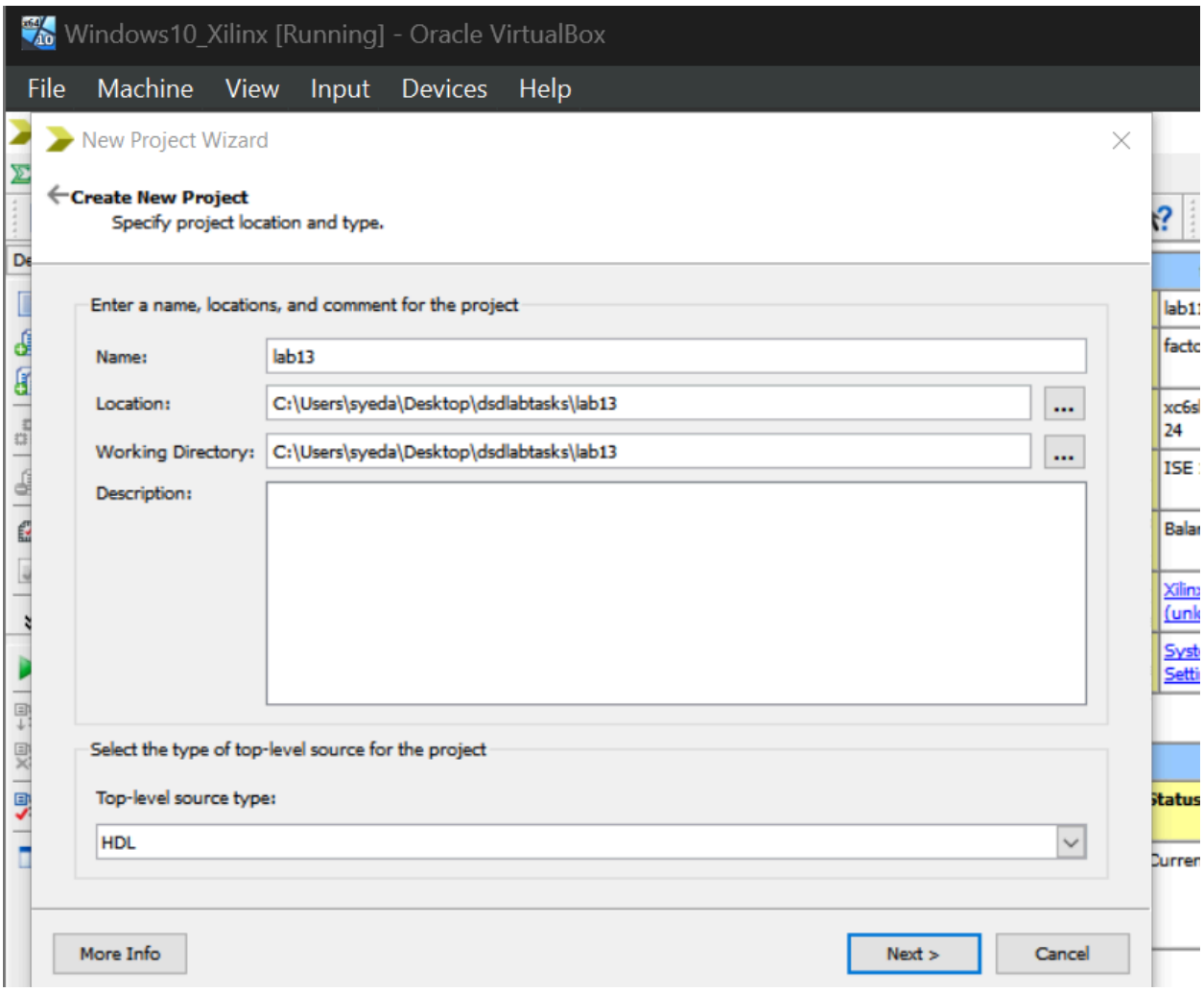
Lab No 13
Syeda AFia Shah
Reg No. 2115

DDR SDRAM

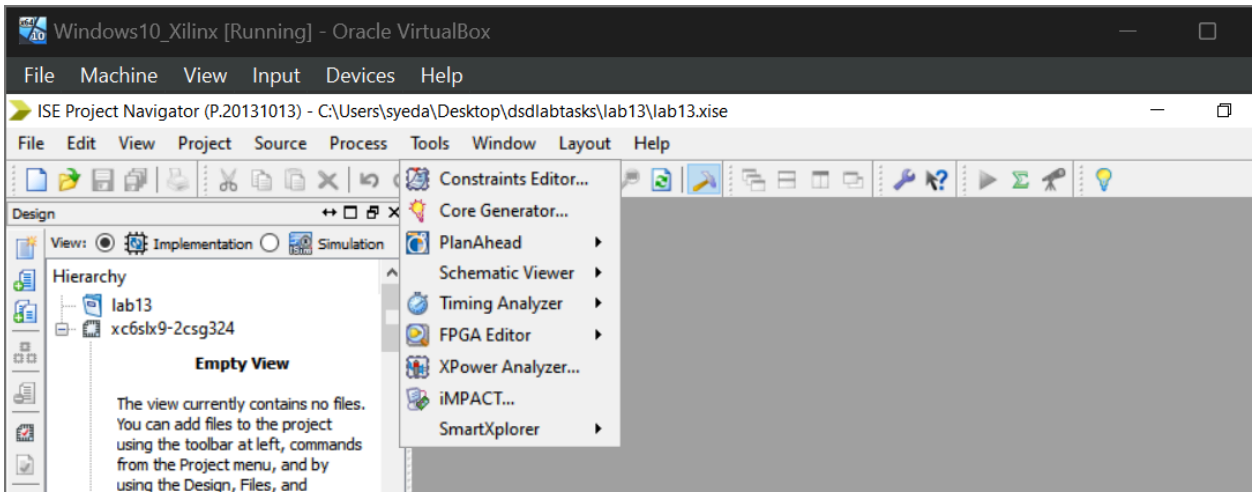
Objective:
To interface and access DDR SDRAM (512Mbit LPDDR) on the Mimas V2 FPGA development board using Xilinx Spartan-6 and Xilinx ISE 14.7 tools.

1. Start Xilinx ISE and select a new project from the File menu. The project wizard will pop up. Type in a project name and path in the first page. Select appropriate FPGA device in the second page. Mimas V2 has a Spartan 6 LX9 device (XC6SLX9). Settings, as shown in the image below, should work fine.

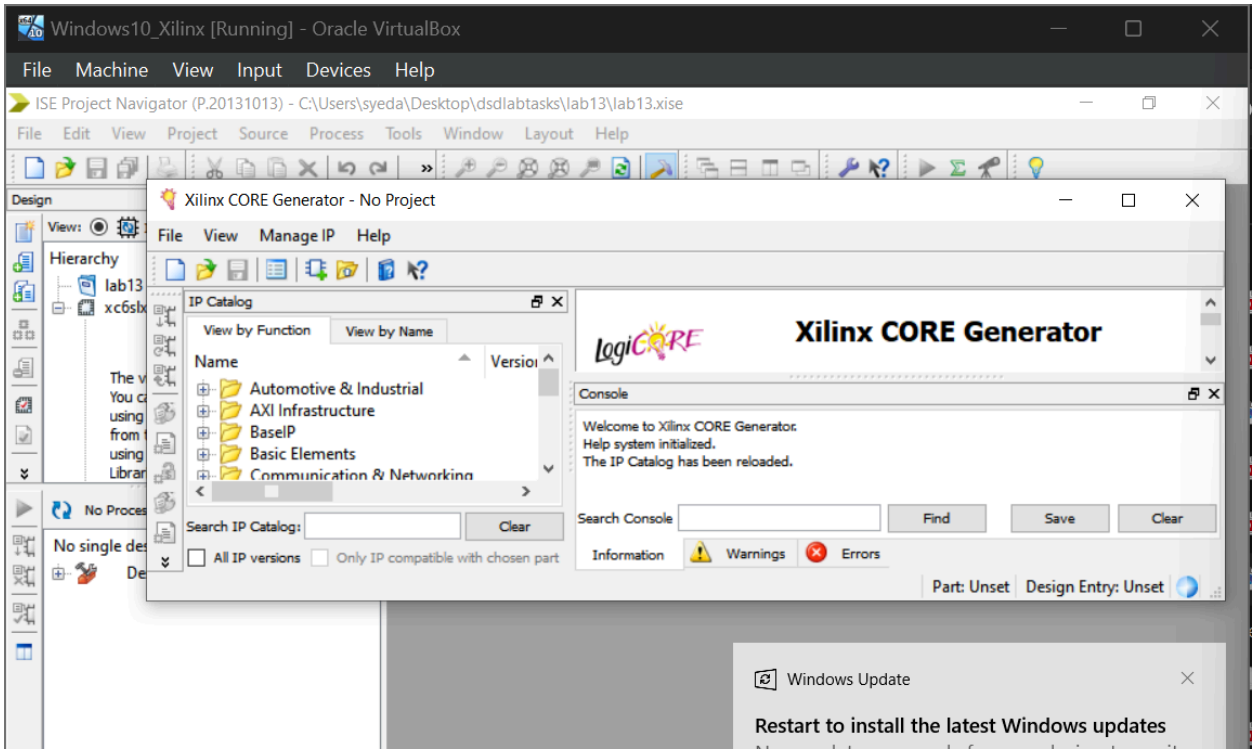


Click next and finish the wizard. Surprisingly we are not going to create or add any source files to the project and our use of ISE's graphical user interface ends once we generate the required IP and other files using Coregen tool. To start

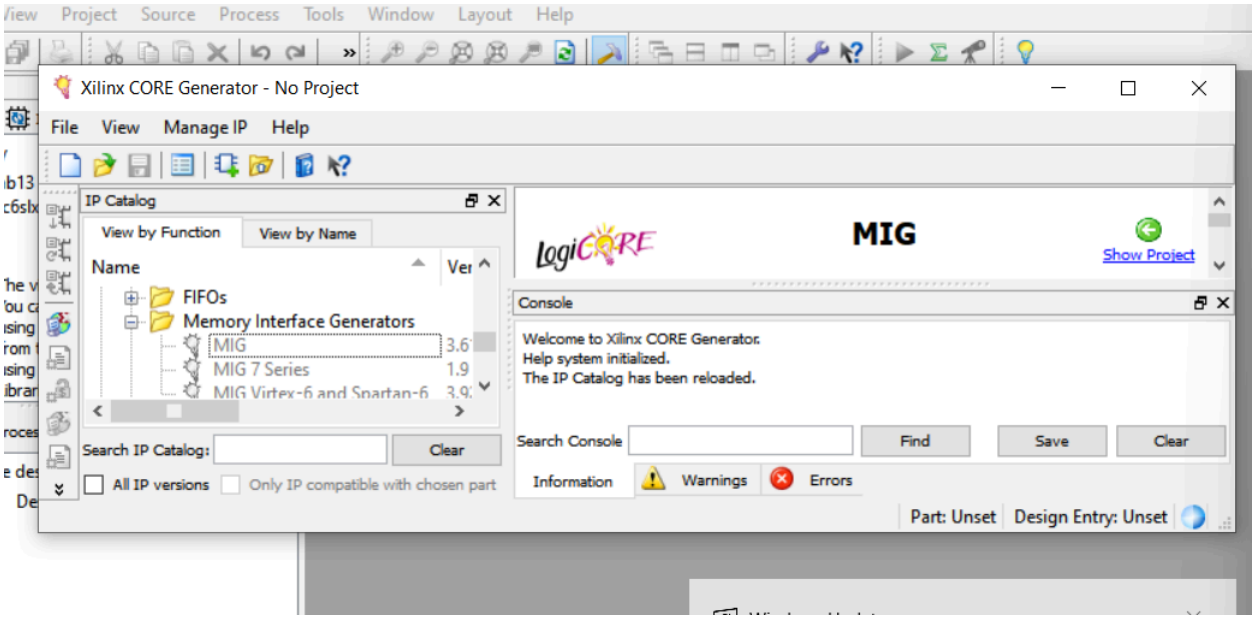
Coregen tool, go to the Tools menu and select “Core Generator”.

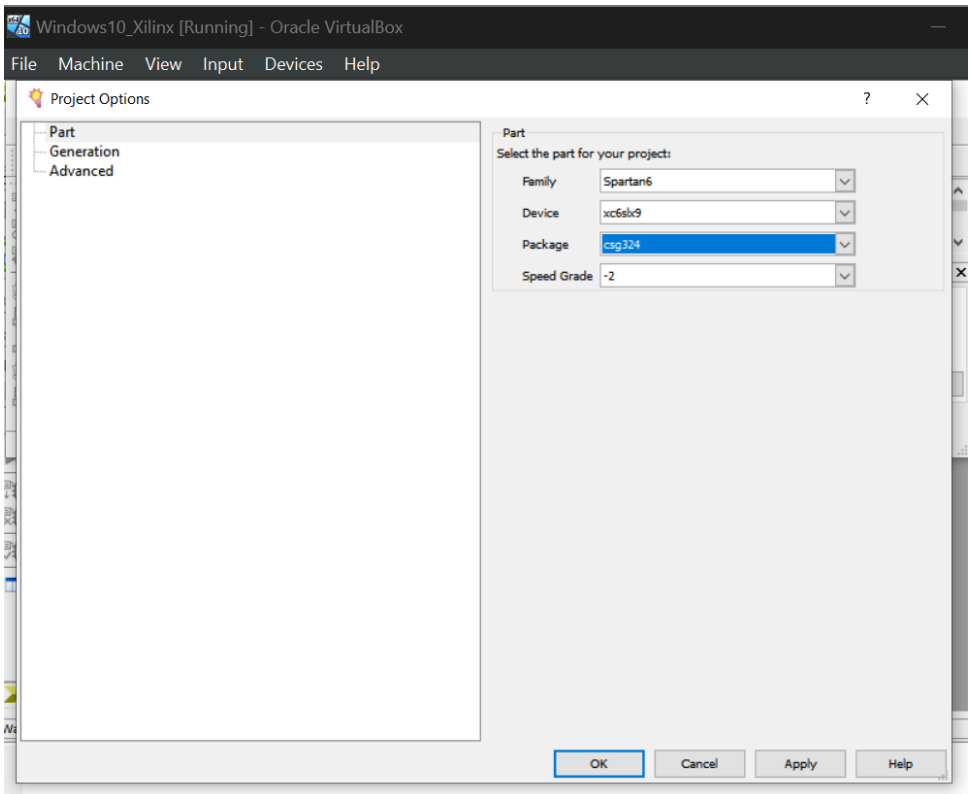


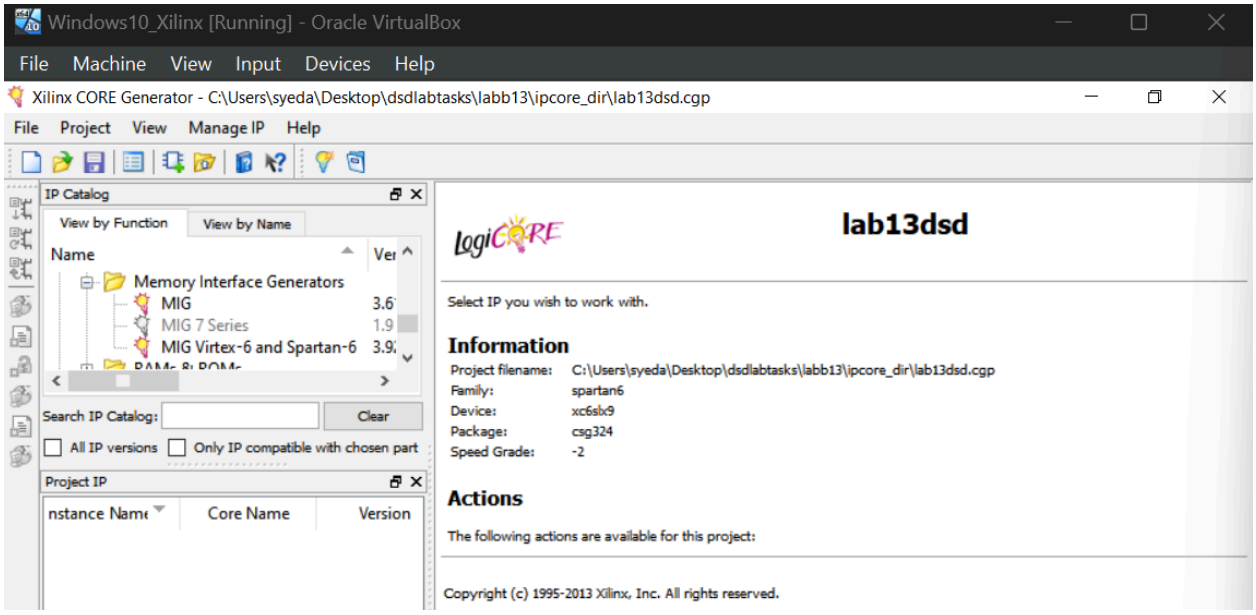
If Core Generator does not create a project automatically, create a project by selecting File>New Project. You will need to select the FPGA and its package when creating the project. Select Verilog as a design entry method. This is to make sure that Core Generator generates code in Verilog. In the IP catalog window, Find MIG under “Memory & Storage Elements” Category.

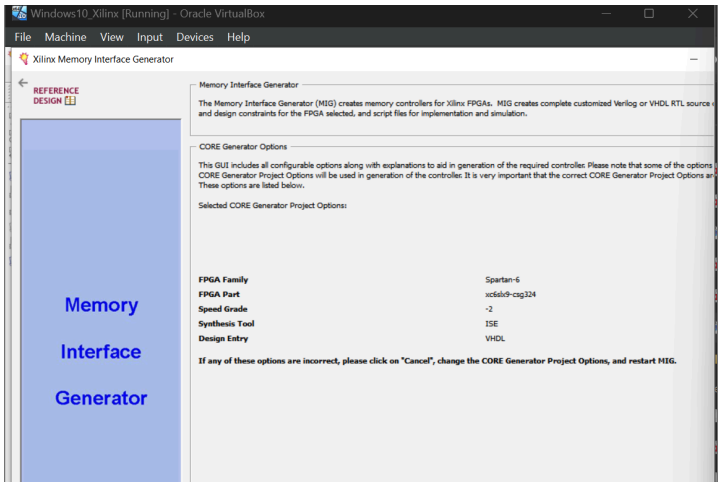


Double click to run Memory Interface Generator wizard. On the first screen, make sure that the selected FPGA device and other settings are correct. The settings should look like in the image below.

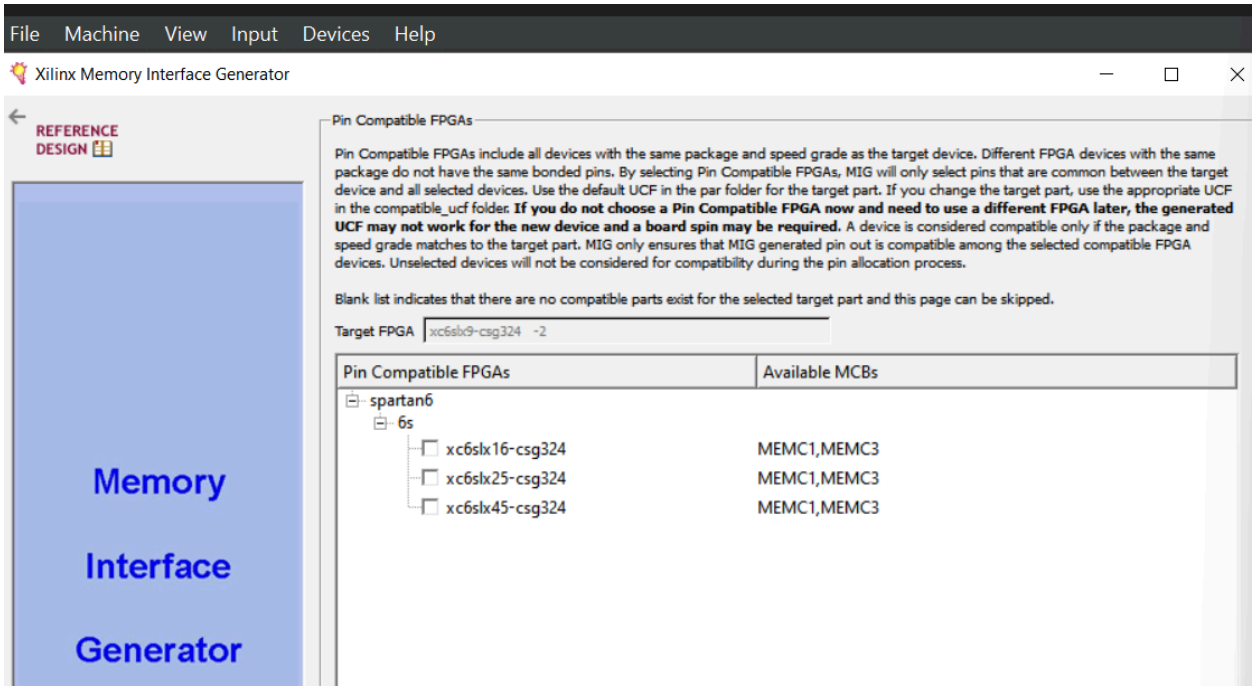

Project Source Process Tools Window Layout Help
Xilinx CORE Generator - No Project
File View Manage IP Help
IP Catalog
View by Function
View by Name
Name
FIFOs
Memory Interface Generators
MIG
MIG 7 Series
Search IP Catalog:
Clear
All IP versions
Only IP compatible with chosen part
MIG
Show Project
Console
Welcome to Xilinx CORE Generator.
Help system initialized.
The IP Catalog has been reloaded.
Search Console
Find
Save
Clear
Information
Warnings
Errors
Part: Unset
Design Entry: Unset


Windows10_Xilinx [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Project Options
Part
Generation
Advanced
Part
Select the part for your project:
Family
Spartan6
Device
xc6slx9
Package
csg324
Speed Grade
-2
OK
Cancel
Apply
Help



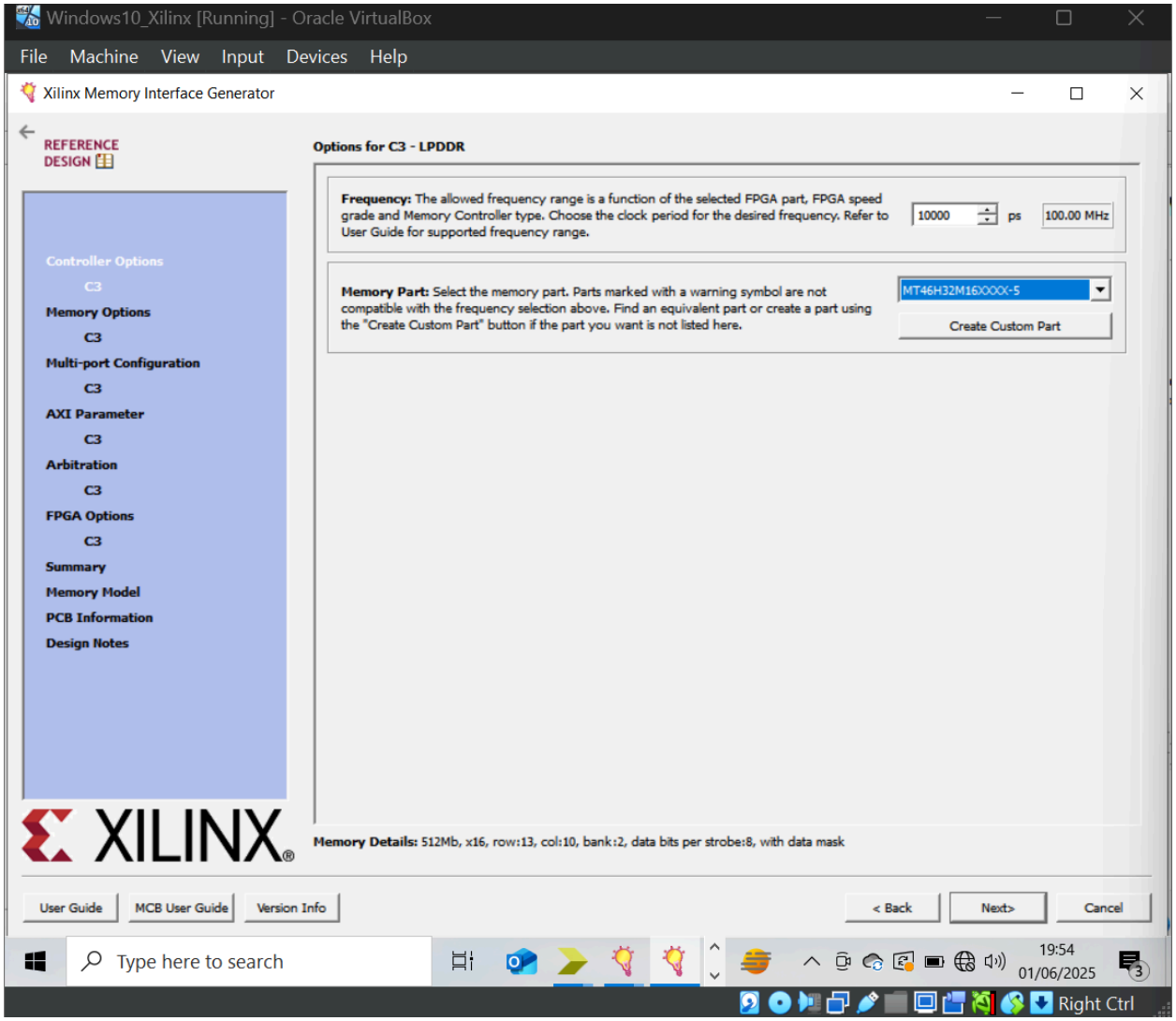


Click next to proceed to the next screen and type in a component name if necessary. Leaving the default name should work fine as well. For the sake of clarity, we will call our component “s6_lpddr”. Please replace this with the component name you chose when “s6_lpddr” appears later in this tutorial.



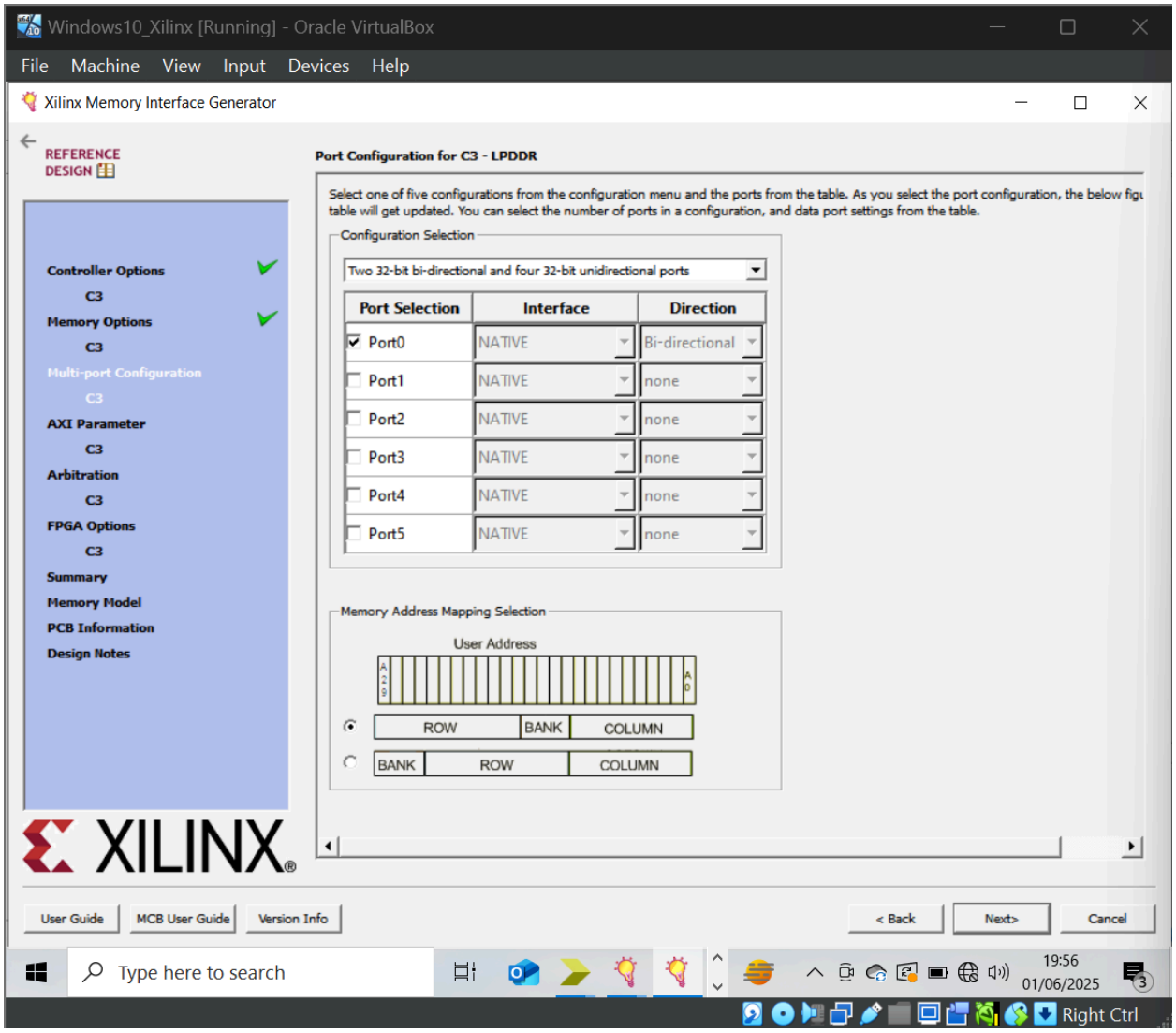
Click next to go to screen three of the wizard. Leave all options unchanged and proceed to screen 4. This screen is where we select the type of DDR memory and tell the MIG wizard where it is connected. Spartan 6 LX9 device has two memory controllers available. On Mimas V2 FPGA Development Board, the LPDDR device is connected to Bank 3 of the FPGA. Select LPDDR from the combo box corresponding to Bank 3. Leave Bank 1 settings unchanged. Settings on this page should look like as in the image below.

Click next to go to the next screen. This is the place where we select the DDR memory device and its operating frequency. Mimas V2 has onboard LPDDR memory which is Micron MT46H32M16 or equivalent. This device supports DDR clock up to 166MHz. Select MT46H32M16 memory device and set the clock period to 10,000. Clock period 10,000 corresponds to 100MHz DDR clock frequency. Though the DDR device supports up to 166MHz clock, we will use 100MHz to avoid the complication of messing with the PLL settings later. Mimas V2 has a 100MHz clock source and by using the same frequency for DDR clock, we can leave the PLL settings generated by MIG as is. Below is the image with correct memory part and frequency selected.



Click next to proceed to the next screen and leave all settings to its defaults. Click next again to proceed to the port configuration screen. Select Port 0 and leave rest of the ports unchecked as shown below.

Click next to proceed to the next screen and leave all settings to its defaults. Click next again to proceed to the port configuration screen. Select Port 0 and leave rest of the ports unchecked as shown below.

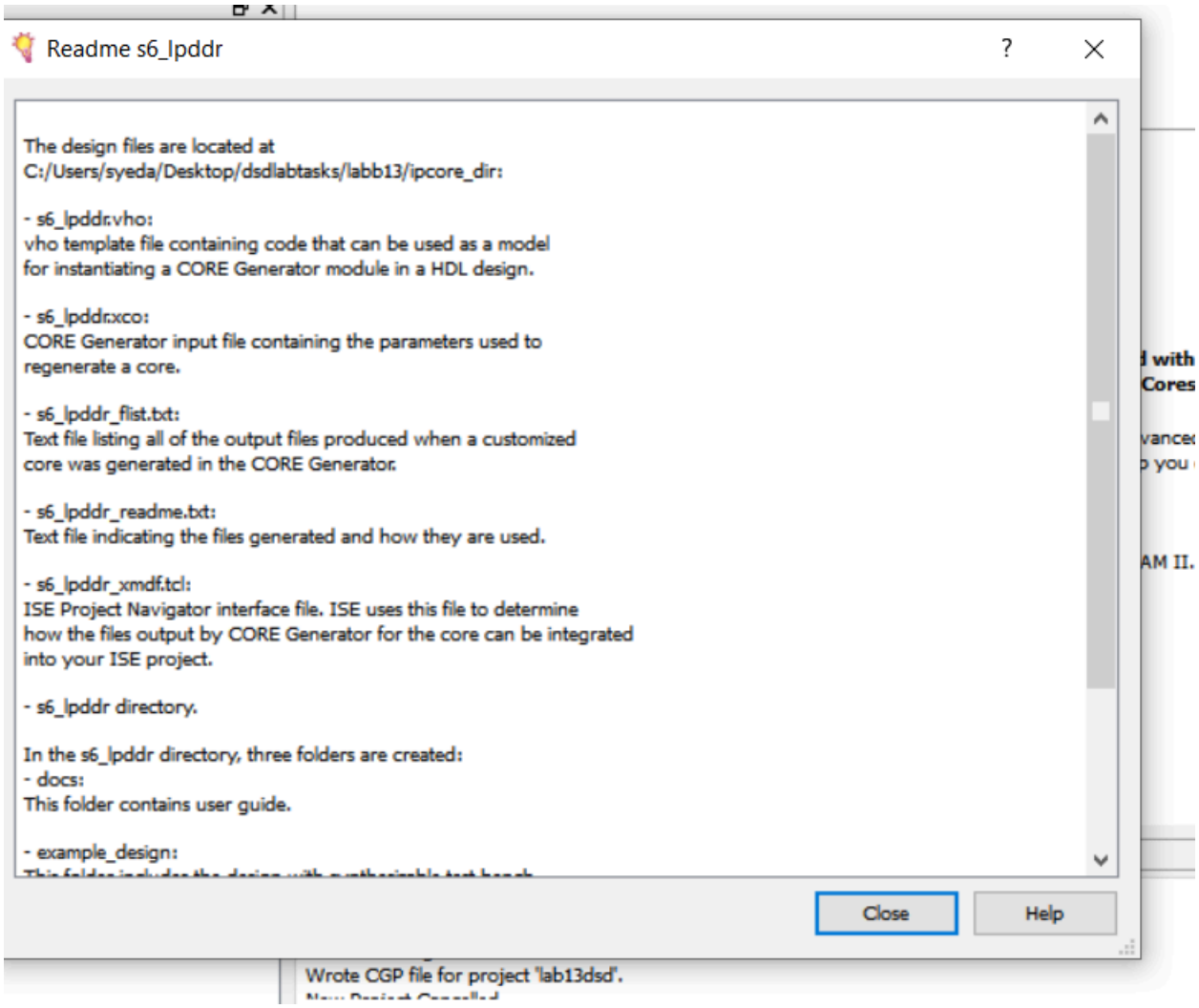


Click next to proceed to the arbitration configuration page. Since we are using only one port, there are no parameters to change on this screen. Click next again to proceed to FPGA Options screen. Select N4 as RZQ pin location and select Single Ended as system clock input. See image below with correct settings.

Click next to proceed to the arbitration configuration page. Since we are using only one port, there are no parameters to change on this screen. Click next again to proceed to FPGA Options screen. Select N4 as RZQ pin location and select Single Ended as system clock input. See image below with correct settings.

Click the next button a few more times and finish. Core Generator will generate a bunch of files. These files can be found under the directory \ipcore_dir\s6_lpddr (Assuming you used the name “s6_lpddr” for the auto-generated component). You will see three folders here, docs, example_design and user_design. docs folder has some very important documentation that can be used to learn more about Spartan 6 Memory Controller and the IP generated by MIG. Keep them for a later read.

Click the next button a few more times and finish. Core Generator will generate a bunch of files. These files can be found under the directory \ipcore_dir\s6_lpddr (Assuming you used the name “s6_lpddr” for the auto-generated component). You will see three folders here, docs, example_design and user_design. docs folder has some very important documentation that can be used to learn more about Spartan 6 Memory Controller and the IP generated by MIG. Keep them for a later read.



Editing user constraints

Right now we are going to use the example design generated by MIG. The example design can be found (unsurprisingly) under the folder example_design. There are a few folders and files inside the example_design folder. rtl folder has all the Verilog files generated by MIG. par folder contains some batch files and scripts to build the example design. The user constraints seem to be auto-generated to match with Xilinx’s own development boards. Some changes are necessary to make the

auto-generated code work with Mimas V2. Before building the project, we need to do the following.

1. Edit the ucf file to make it usable with Mimas V2
2. Configure the build environment to generate a binary configuration file

Go to par folder and find example_top.ucf. Open example_top.ucf using any text editor of your choice. Make the following changes.

1. Change the line CONFIG VCCAUX=2.5; to CONFIG VCCAUX=3.3; . This change is necessary because Mimas V2 uses 3.3V for VCCAUX.
2. Change the following lines
   NET “error” IOSTANDARD = LVCMOS18;
   NET “calib_done” IOSTANDARD = LVCMOS18;
   NET “calib_done” LOC = “B2” ;
   NET “error” LOC = “A2” ;ToNET “error” IOSTANDARD = LVCMOS33;
   NET “calib_done” IOSTANDARD = LVCMOS33;
   NET “calib_done” LOC = “T18” ; #LED1
   NET “error” LOC = “T17” ; #LED2

Above changes will make the “error” and “calib_done” pins operate at LVCMOS33 IO standard and assign the nets to T18 and T17 of FPGA where LED1 and LED2 are connected. This will cause LED1 to turn on when calibration is done and LED2 to turn on if memory test failed

3. Change the linesNET “c3_sys_clk” IOSTANDARD = LVCMOS25;
   NET “c3_sys_rst_n” IOSTANDARD = LVCMOS18;ToNET “c3_sys_clk” IOSTANDARD = LVCMOS33;
   NET “c3_sys_rst_n” IOSTANDARD = LVCMOS33;

Above change will set the IO standards for clock input and reset input to LVCMOS33. This is again because the bank that these IO belongs is powered by 3.3V rail.

4. Add a line NET “c3_sys_rst_n” PULLDOWN;

This will enable pull down on the reset pin and will keep the memory controller our of reset without having to use any external components. Despite the name “c3_sys_rst_n”, MIG seems to be configuring reset input as active high.

5. Change “c3_sys_clk” and “c3_sys_rst_n” pin assignments as below.
   NET “c3_sys_clk” LOC = “V10” ;
   NET “c3_sys_rst_n” LOC = “M16” ;

This will assign correct IO pads for clock input and assign switch SW3 as reset input. We are done with the changes in ucf file now. This may seem difficult but easy enough if done carefully. It is a good idea to back up your original ucf file before saving the changes just in case if you want to go back and restart again.

Building the code

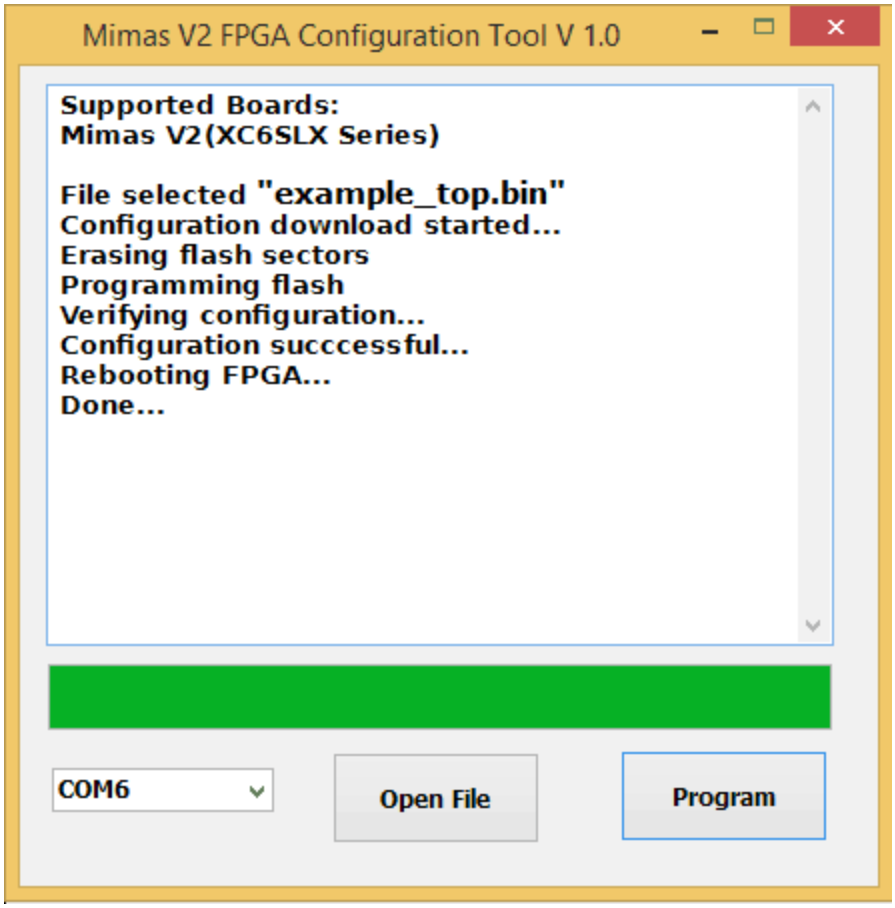
Next step is to modify the build environment to generate a binary configuration file. This is a very easy step to do. Find the file mem_interface_top.ut and open it in a text editor. Find the line “-g Binary:no” and change it to “-g Binary:yes” and save. Now we are ready to build the project. Before building the project, make sure that the path to Xilinx build tools is added to PATH environment variable. Usually, the path is C:\Xilinx\\ISE_DS\ISE\bin\nt assuming ISE is installed on C: drive.
Now run the batch file ise_flow.bat by double-clicking the file or by using the command prompt. If everything went fine so far, the batch file will run the necessary tools to build the project and you will end up with a “Done” message and a bunch of new files in the par folder. The message should look like in the image below.

```
C:\ddrdemo\ddrdemo\ipcore_dir\s6_lpddr\example_design\par>par -w -intstyle

C:\ddrdemo\ddrdemo\ipcore_dir\s6_lpddr\example_design\par>trce -e 100 exam

C:\ddrdemo\ddrdemo\ipcore_dir\s6_lpddr\example_design\par>bitgen -intstyle

C:\ddrdemo\ddrdemo\ipcore_dir\s6_lpddr\example_design\par>echo done!
done!
```

If the build process fails, refer to ise_flow_results.txt for more details on the causes of failure. You should see the file “example_top.bin” in par folder if build succeeded. This is the file we are going to program Mimas V2 Spartan 6 FPGA development board with.

Configuring Mimas V2 and testing

Programming Mimas V2 Spartan 6 FPGA development board is very easy. Download the latest version of the Configuration Downloader Application from the product page. Run the executable, no installation required. Load the binary file and program the flash as illustrated in the image below.



Editing user constraints

Right now we are going to use the example design generated by MIG. The example design can be found (unsurprisingly) under the folder example_design. There are a few folders and files inside the example_design folder. rtl folder has all the Verilog files generated by MIG. par folder contains some batch files and scripts to build the example design. The user constraints seem to be auto-generated to match with Xilinx's own development boards. Some changes are necessary to make the auto-generated code work with Mimas V2. Before building the project, we need to do the following.

1. Edit the ucf file to make it usable with Mimas V2
2. Configure the build environment to generate a binary configuration file

Go to par folder and find example_top.ucf. Open example_top.ucf using any text editor of your choice. Make the following changes.

1. Change the line CONFIG VCCAUX=2.5; to CONFIG VCCAUX=3.3; . This change is necessary because Mimas V2 uses 3.3V for VCCAUX.
2. Change the following lines
   NET “error” IOSTANDARD = LVCMOS18;
   NET “calib_done” IOSTANDARD = LVCMOS18;
   NET “calib_done” LOC = “B2” ;
   NET “error” LOC = “A2” ;ToNET “error” IOSTANDARD = LVCMOS33;
   NET “calib_done” IOSTANDARD = LVCMOS33;
   NET “calib_done” LOC = “T18” ; #LED1
   NET “error” LOC = “T17” ; #LED2

Above changes will make the “error” and “calib_done” pins operate at LVCMOS33 IO standard and assign the nets to T18 and T17 of FPGA where LED1 and LED2 are connected. This will cause LED1 to turn on when calibration is done and LED2 to turn on if memory test failed

3. Change the linesNET “c3_sys_clk” IOSTANDARD = LVCMOS25;
   NET “c3_sys_rst_n” IOSTANDARD = LVCMOS18;ToNET “c3_sys_clk” IOSTANDARD = LVCMOS33;
   NET “c3_sys_rst_n” IOSTANDARD = LVCMOS33;

Above change will set the IO standards for clock input and reset input to LVCMOS33. This is again because the bank that these IO belongs is powered by 3.3V rail.

4. Add a line NET “c3_sys_rst_n” PULLDOWN;

This will enable pull down on the reset pin and will keep the memory controller our of reset without having to use any external components. Despite the name “c3_sys_rst_n”, MIG seems to be configuring reset input as active high.

5. Change “c3_sys_clk” and “c3_sys_rst_n” pin assignments as below.
   NET “c3_sys_clk” LOC = “V10” ;
   NET “c3_sys_rst_n” LOC = “M16” ;

This will assign correct IO pads for clock input and assign switch SW3 as reset input. We are done with the changes in ucf file now. This may seem difficult but easy enough if done carefully. It is a good idea to back up your original ucf file before saving the changes just in case if you want to go back and restart again.

```
# Status Signals
##############################################################################

NET  "error"                                    IOSTANDARD = LVCMOS33;
NET  "calib_done"                               IOSTANDARD = LVCMOS33;
NET  "calib_done"                               LOC = "T18" ;
NET  "error"                                    LOC = "T17" ;

##############################################################################
```


```
# Status Signals
##############################################################################

NET  "error"                                    IOSTANDARD = LVCMOS33;
NET  "calib_done"                               IOSTANDARD = LVCMOS33;
NET  "calib_done"                               LOC = "T18" ;
NET  "error"                                    LOC = "T17" ;

##############################################################################
```


```
NET  "mcb3_dram_udm"                              IOSTANDARD = MOBILE_DDR;
NET  "mcb3_rzq"                                   IOSTANDARD = MOBILE_DDR;
NET  "c3_sys_clk"                               IOSTANDARD = LVCMOS33;
NET  "c3_sys_rst_n"                             IOSTANDARD = LVCMOS33;
NET  "c3_sys_rst_n"                             PULLDOWN;
##############################################################################
```

Building the code

Next step is to modify the build environment to generate a binary configuration file. This is a very easy step to do. Find the file mem_interface_top.ut and open it in a text editor. Find the line “-g Binary:no” and change it to “-g Binary:yes” and save.

*mem_interface_top - Notepad

File Edit Format View Help

```
-w
-g DebugBitstream:No
-g Binary:yes
-g CRC:Enable
-g M2Pin:PullUp
-g ProgPin:PullUp
-g DonePin:PullUp
```

Now we are ready to build the project. Before building the project, make sure that the path to Xilinx build tools is added to PATH environment variable. Usually, the path is C:\Xilinx\\ISE_DS\ISE\bin\nt assuming ISE is installed on C: drive.

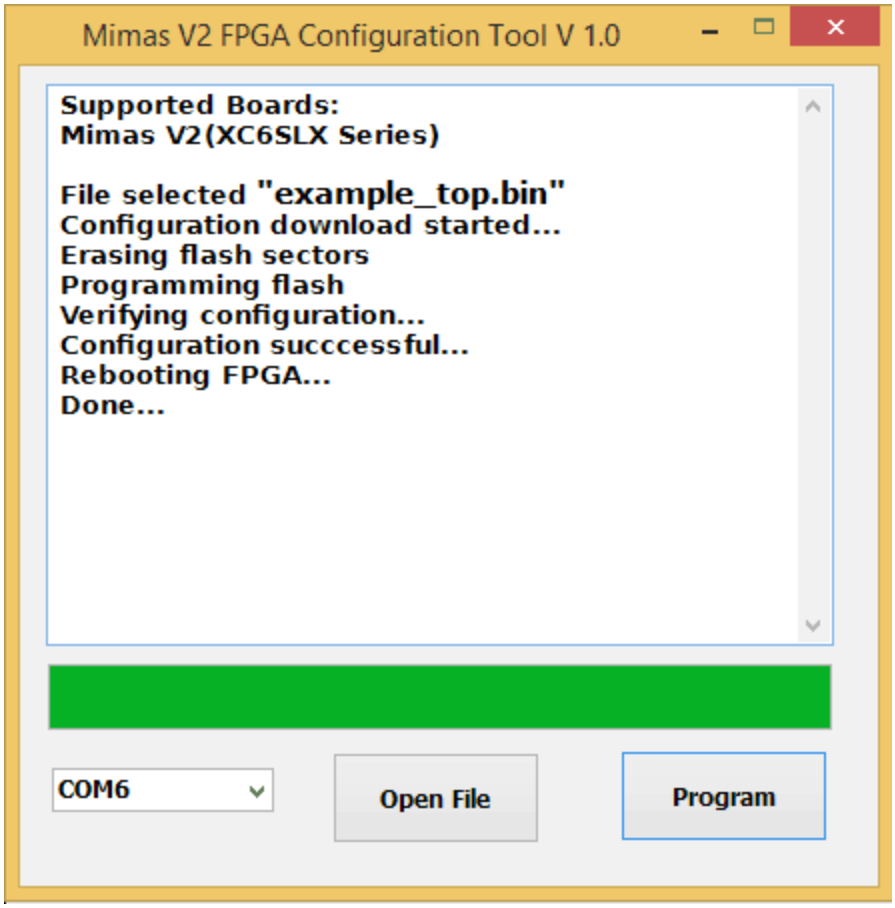
Now run the batch file ise_flow.bat by double-clicking the file or by using the command prompt. If everything went fine so far, the batch file will run the necessary tools to build the project and you will end up with a “Done” message and a bunch of new files in the par folder. The message should look like in the image below.

```
C:\ddrdemo\ddrdemo\ipcore_dir\s6_lpddr\example_design\par>par -w -intstyle

C:\ddrdemo\ddrdemo\ipcore_dir\s6_lpddr\example_design\par>trce -e 100 exam

C:\ddrdemo\ddrdemo\ipcore_dir\s6_lpddr\example_design\par>bitgen -intstyle

C:\ddrdemo\ddrdemo\ipcore_dir\s6_lpddr\example_design\par>echo done!
done!
```

If the build process fails, refer to ise_flow_results.txt for more details on the causes of failure. You should see the file “example_top.bin” in par folder if build succeeded. This is the file we are going to program Mimas V2 Spartan 6 FPGA development board with.

Configuring Mimas V2 and testing

Programming Mimas V2 Spartan 6 FPGA development board is very easy. Download the latest version of the Configuration Downloader Application from the product page. Run the executable, no installation required. Load the binary file and program the flash as illustrated in the image below.


Mimas V2 FPGA Configuration Tool V 1.0
Supported Boards:
Mimas V2(XC6SLX Series)
File selected "example_top.bin"
Configuration download started...
Erasing flash sectors
Programming flash
Verifying configuration...
Configuration succcessful...
Rebooting FPGA...
Done...
COM6
Open File
Program